

C TYPE CONVERSION

IMPLICIT & EXPLICIT TRANSFORMATIONS



The Complete 18-Page Programmer's Manual

01. Introduction

Data type conversion in C is the mechanism by which a value is shifted from one data type to another. This is a fundamental operation in languages that are **Statically Typed** like C.

Definition: *Conversion is the process of changing a variable of one data type into another type during program execution.*

C allows seamless conversion between compatible types like integers, floating-points, and characters.

02. Why Conversion Matters?

In a perfect world, all variables in an expression would be the same type. However, real-world data is messy.

- **Mixed Expressions:** Calculating `int` values with `float` values.
- **Assignment:** Storing a `char` result into an `int` container.
- **Accuracy:** Forcing decimal results from integer division.
- **Logic Alignment:** Ensuring data fits the function parameters required.

03. Classification

C provides two distinct paths for type conversion:

1. Implicit Conversion

Automatic conversion by the compiler (Promotion).

2. Explicit Conversion

Manual conversion by the programmer (Type Casting).

04. Implicit Type Conversion

Also known as **Automatic Conversion**. This occurs when the compiler converts a "smaller" data type to a "larger" one to ensure the operation succeeds without data loss.

```
int a = 10;  
float b = a; // Compiler converts 'a' to 10.0 automatically
```

This is safe because a float can represent any integer value without loss.

05. ASCII Implicit Conversion

In C, characters and integers are cousins. You can assign a char to an int without a cast.

```
char ch = 'A';  
int i = ch; // 'i' is now 65
```

The compiler looks up the ASCII table and stores the numeric representative.

06. The Promotion Ladder

C follows a strict hierarchy for automatic promotion to ensure precision stays high:

char → int → float → double → long double

In an expression like `5 + 2.5`, the `int 5` is promoted to `float 5.0`, resulting in `7.5`.

07. Explicit Type Conversion

When you need a conversion that the compiler won't do (or won't do safely), you use **Type Casting**.

```
float x = 9.8;  
int y = (int)x; // Manually forcing float to int
```

The syntax (type) is a unary operator that has higher precedence than most math operators.

08. The Division Trap

By default, `int / int` always results in an `int`. This leads to common logical bugs.

```
int a = 5, b = 2;  
float res = a / b; // Result: 2.0 (Wrong!)
```

Even though the result is stored in a float, the math was done in integer mode first.

09. Solving with Casting

To fix the division trap, cast at least one operand to a float.

```
float res = (float)a / b; // Result: 2.5 (Correct!)
```

Now, the compiler promotes the entire expression to floating-point math.

10. Truncation vs Rounding

When you cast a float to an int, C does **Truncation**, not rounding.

```
int val1 = (int)3.14; // Result: 3  
int val2 = (int)3.99; // Result: 3
```

The fractional part is simply discarded, regardless of how close it is to the next whole number.

11. Integer Promotion

In C, small types (`char`, `short`) are automatically promoted to `int` when used in any mathematical expression.

```
char a = 10, b = 20;  
int c = a + b; // 'a' and 'b' become ints for the sum
```

12. The Danger of "Narrowing"

Converting from double to int or long to short is called narrowing. This can lead to:

- **Loss of Precision:** Decimals gone.
- **Overflow:** Large numbers becoming garbage values because they don't fit the smaller bit-width.

13. Input/Output Sync

Using `scanf` and `printf` requires specifiers that match the data type. A mismatch can lead to "Undefined Behaviour".

```
int x = 65;  
printf("%c", (char)x); // Prints 'A'
```

14. Real-World Case Study

Calculating the average of three exam scores:

```
int s1=80, s2=85, s3=90;  
float avg = (float)(s1+s2+s3) / 3;  
printf("Avg: %.2f", avg);
```

The cast ensures the percentage is accurate to the decimal point.

15. Top Mistakes

- **Mismatched Parentheses:** `(float)(a/b)` vs `(float)a/b`.
- **Implicit Confusion:** Expecting `int x = 5/2;` to result in 2.5.
- **Specifier Error:** Using `%d` for a variable you just cast to a float.

16. FAQ for Interviews

Q: Difference between Implicit and Explicit?

A: Implicit is compiler-driven; Explicit is programmer-driven.

Q: What is type promotion?

A: Converting smaller types to larger types in expressions to maintain precision.

17. Critical Summary

Direction	Process	Result
Lower → Higher	Implicit	Lossless
Higher → Lower	Explicit	Lossy

END OF DATA TYPE CONVERSION GUIDE